

BIPADLE WALKER

TEAM MINIONS:

Hamsika Parnandi [RA2311047010026]

Darshita Udani [RA2311047010041]

Mohammad Affreen [RA2311047010045]

OVERVIEW

- Abstract
- Introduction
- Literature Survey
- Metrics Used
- Lab Experiment Mapping
- Proposed Solution
- Implementation
- Code
- Results and Conclusion



ABSTRACT

- Reinforcement Learning (RL) is a machine learning technique where an agent learns optimal actions by interacting with an environment and receiving rewards.
- Bipedal locomotion is a complex control problem because it requires balance, coordination of joints, and adaptation to uneven terrain.
- The Bipedal Walker environment is a standard reinforcement learning benchmark used to study locomotion control in simulated robots.
- In this environment, the agent controls a robot with four motorized joints (two hips and two knees) and must learn to walk across rough terrain.
- The agent receives rewards for moving forward and penalties for falling or excessive energy usage.
- The objective of this project is to train an RL agent that can learn stable walking behavior through interaction with the environment.
- The performance of the trained agent is evaluated using metrics such as average reward, episode length, walking stability, and training convergence.
- This project demonstrates how reinforcement learning can learn locomotion policies without explicitly programming gait patterns.

INTRODUCTION

- Robotics locomotion is a challenging problem in artificial intelligence because it requires balance, coordination of multiple joints, and adaptation to different terrains.
- Traditional control methods such as PID controllers and dynamic models often struggle to handle complex and unpredictable environments.
- Reinforcement Learning (RL) allows robots to learn behaviors through trial and error by interacting with the environment and maximizing cumulative rewards.
- The Bipedal Walker environment is commonly used to evaluate RL algorithms for continuous control tasks where a robot learns to walk without falling.
- In this project, an RL agent is trained to control four joint torques (hips and knees) using environmental observations to achieve stable and efficient walking behavior.

LITERATURE SURVEY

Paper Title, Authors & Source	Methodology	Gap Identified	Results & Metrics
Reinforcement Learning of Bipedal Walking Using a Simple Reference Motion – Naoya Itahashi, Hideaki Itoh, Hisao Fukumoto, Hiroshi Wakuya – <i>Applied Sciences (MDPI)</i> , 2024	Reinforcement learning combined with reference motion trajectories to guide walking behavior in humanoid robots.	Requires predefined reference motions which limit exploration of natural gaits.	Metrics: Real robot walking speed: 1.9 cm/s Simulation walking speed: 1.79 ± 0.23 cm/s Stride length: 3.8 cm Single Support Phase (SSP): 1.92 s Whole gait cycle: 2.00 s
Emergence of Natural and Robust Bipedal Walking by Learning from Biologically Plausible Objectives – Research team in iScience (Elsevier), 2025	Uses biologically inspired reward functions with reinforcement learning to train musculoskeletal walking models.	High computational cost due to complex biomechanical models.	Metrics: Trained on 4 musculoskeletal models in 2D/3D; best gait quality achieved on H1622 model (16 DOF, 22 muscles); walking speed 1.2 m/s; running speed 5.2 m/s; outperformed
Learning Bipedal Walking for Humanoid Robots in Challenging Environments with Obstacle Avoidance – Marwan Hamze, Mitsuharu Morisawa, Eiichi Yoshida – J-STAGE Robotics Conference,	Policy-based deep reinforcement learning with additional reward terms for obstacle avoidance.	Previous RL methods mainly focus on flat terrain environments.	Metrics: Average return = 138; Episode length = 400 iterations; Training samples = 96 million; Training time = 10 hours; Most runs completed full episode successfully
Template Model Inspired Task Space Learning for Robust Bipedal Locomotion – Guillermo A. Castillo, Bowen Weng, Shunpeng Yang, Wei Zhang, Ayonga Hereid – arXiv, 2023	Hierarchical RL framework with high-level RL planner and low-level model-based controller.	Hybrid architecture increases system complexity.	Metrics: Tracks speeds -1 to 1 m/s (Rabbit/Walker2D); Digit handles vx = -1.0 to 1.5 m/s, vy = -0.5 to 0.5 m/s; stable on slopes up to 20° (Rabbit/Walker2D) and 10° (Digit); resisted disturbances of -80 to 60 N for 0.15–3 s without falling

Infer and Adapt: Bipedal Locomotion Reward Learning from Demonstrations via Inverse Reinforcement Learning – Feiyang Wu, Zhaoyuan Gu, Hanran Wu, Anqi Wu, Ye Zhao – <i>arXiv</i> , 2023	Uses inverse reinforcement learning to learn reward functions from expert demonstrations.	Requires expert demonstrations which may not always be available.	Metrics: Adaptive reward outperformed baseline in sagittal distance: 10.5 vs 4.5 (perturbed), 14.0 vs 6.5 (gradual), 15.5 vs 7.5 (gravel) at 600 steps; optimizer Adam, learning rate 3×10^{-4} , reward approximation trained using MSE
Reinforcement Learning for Versatile, Dynamic, and Robust Bipedal Locomotion Control – Zhongyu Li, Xue Bin Peng, Pieter Abbeel, Sergey Levine, Glen Berseth, Koushil Sreenath – <i>arXiv</i> , 2024	End-to-end deep RL controller with dual-history architecture for locomotion skills.	Requires large training data and simulation resources.	Metrics: Feasible command set $>4\times$ larger than HZD baseline; stable walking speed -1.2 to 1.2 m/s; minimum walking height 0.65 m; handled 2.0 m/s command with achieved speed 1.2 m/s; robust under perturbation intensity $\beta = 0.8-1.0$
Hybrid Bipedal Locomotion Based on Reinforcement Learning and Heuristics – Zhicheng Wang, Wandi Wei, Anhuan Xie, Yifeng Zhang, Jun Wu, Qiuguo Zhu – <i>Micromachines</i> (MDPI), 2022	Combines heuristic controllers with reinforcement learning for locomotion planning.	Hybrid approach reduces flexibility and learning generalization.	Metrics: Tracked velocity up to 2 km/h (0.556 m/s); tracking MSE 2.232×10^{-3} vs baseline 1.282×10^{-2} ; robust to ± 5 kg torso variation; gait frequency 2.1 Hz; recovered from 14 N push for 0.5 s while walking at 0.6 m/s

Standing Balance of Single-Legged Hopping Robot Model using Reinforcement Learning – S. Mohamad Hoseinifard, Majid Sadedel – Scientific Reports (Nature), 2024	Reinforcement learning applied for balance control under external disturbances.	Focuses on single-leg robot models rather than full biped locomotion.	Metrics: Validated with 22 cm drop test; trained for 4650 episodes with average reward 100,000; jump height 42 cm; maintained balance for 40 s while traveling 8 m; recovered from 25 N / 0.1 s impact, failed at 32 N / 0.1 s; tolerated 3 cm ground height drop
RL-Augmented MPC Framework for Agile and Robust Bipedal Footstep Planning – Seung Hyeon Bang, Carlos Arribalzaga Jové, Luis Sentis – arXiv, 2024	Combines model predictive control with reinforcement learning for footstep planning.	Requires accurate robot dynamic models.	Metrics: Lateral velocity tracking -0.83 to 0.83 m/s vs baseline -0.6 to 0.6 m/s; turning rate 1.27 rad/s with 2.7 full turns/episode (350% better than baseline); low-frequency policy tracked 1.9 rad/s; resisted pushes of 35 N (1 s) and 350 N (0.0175 s); stable on 11.5° slopes
Data-Driven Latent Space Representation for Robust Bipedal Locomotion Learning – Guillermo A. Castillo, Bowen Weng, Wei Zhang, Ayonga Hereid – arXiv, 2023	Uses autoencoder-based latent space representation combined with RL locomotion policy.	Requires large locomotion datasets for training representation.	Metrics: Latent dimensions tested N = 2, 4, 8; tracked speeds -0.75 to 1.4 m/s using Digit robot; training data only -0.5 to 1.0 m/s (shows generalization); outperformed ALIP planner which failed above 1.2 m/s; robust to disturbances of -100 to 60 N for 0.1–1.5 s

METRICS USED

1.AVERAGE REWARD:

- MEAN REWARD OBTAINED PER EPISODE.
- INDICATES OVERALL LEARNING PERFORMANCE.

2.EPISODE LENGTH:

- NUMBER OF TIME STEPS THE AGENT SURVIVES BEFORE FALLING.

3.WALKING SPEED:

- DISTANCE TRAVELED PER EPISODE.

4.ENERGY CONSUMPTION:

- AMOUNT OF MOTOR TORQUE USED DURING LOCOMOTION.

5.TRAINING CONVERGENCE

- NUMBER OF EPISODES REQUIRED FOR THE MODEL TO STABILIZE.

6.SUCCESS RATE

- PERCENTAGE OF EPISODES WHERE THE ROBOT REACHES THE END OF THE TERRAIN.

7.STABILITY SCORE

MEASURES HOW WELL THE ROBOT MAINTAINS BALANCE.

LAB EXPERIMENT MAPPING

Experiment Component	Description
Problem Statement	Train a reinforcement learning agent to learn stable walking behavior in the Bipedal Walker environment
Environment	OpenAI Gym / Gymnasium BipedalWalker-v3
State Space	24-dimensional observation including hull angle, velocity, joint angles, ground contact sensors, and lidar readings
Action Space	Continuous control of four joints (two hips and two knees)
RL Algorithm	Proximal Policy Optimization (PPO) / Actor-Critic
Reward Design	Positive reward for forward movement, penalty for falling, penalty for excessive motor torque
Training Setup	Agent interacts with simulated environment through episodic learning
Tools Used	Python, OpenAI Gym, Stable Baselines3, PyTorch
Evaluation Method	Average reward, stability score, episode length
Output	Trained agent capable of stable locomotion across terrain

PROPOSED SOLUTION

1. Simulation Environment Setup

The Bipedal Walker environment from OpenAI Gym / Gymnasium will be used as the simulation platform where the robot learns to walk on uneven terrain.

2. State Observation Input

The agent receives 24 observation values such as robot angle, velocity, joint angles, ground contact sensors, and lidar readings to understand the environment.

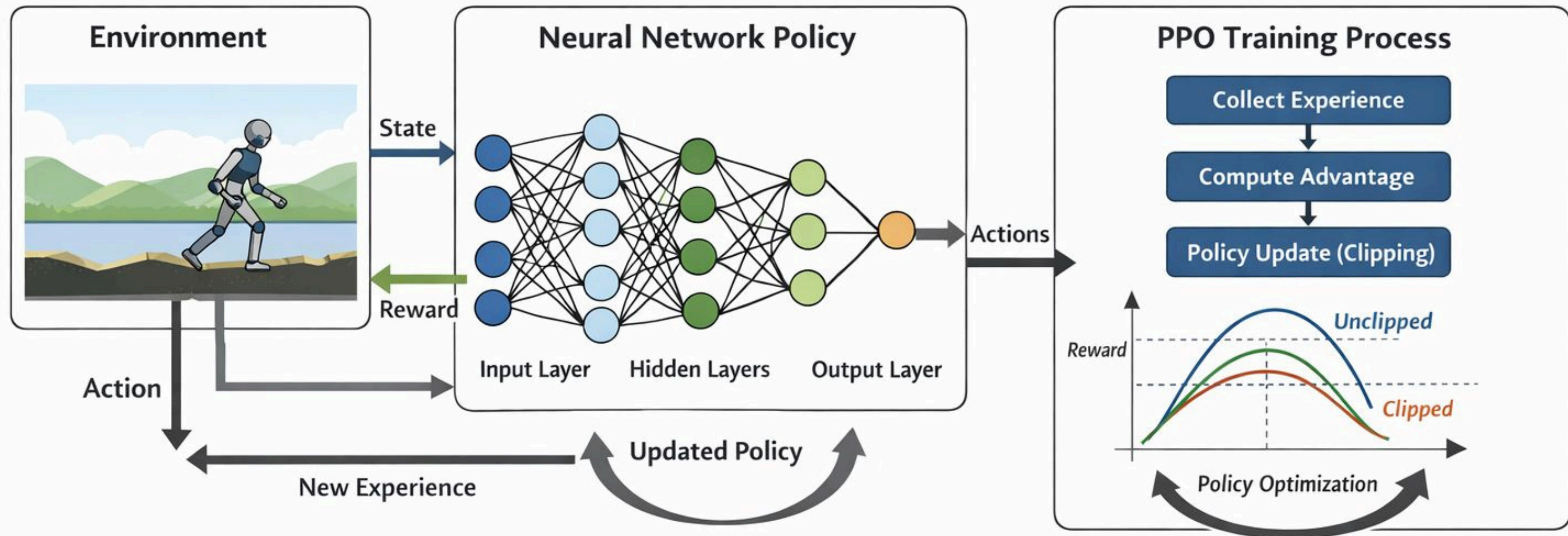
3. Policy Network Implementation

A neural network policy model will be implemented to map the observed state to the appropriate actions for controlling the robot joints.

4. Performance Evaluation

The trained agent will be evaluated based on average reward, episode length, and walking stability to determine the effectiveness of the learning model.

ARCHITECTURE DIAGRAM



IMPLEMENTATION

1. Step 1: Environment Setup

- Installed required libraries:
- Gymnasium
- Stable-Baselines3
- Imported necessary modules for RL training and video recording

2. Step 2: Environment Initialization

- Created the BipedalWalker-v3 simulation environment
- Used `reset()` to initialize the robot and terrain
- Verified state space (observations) and action space (joint torques)

3. Step 3: Model Selection and Initialization

- Selected Proximal Policy Optimization (PPO)
- Used MLP (Neural Network Policy) for mapping states → actions
- Initialized model with hyperparameters like learning rate, batch size, discount factor (gamma).

4. **Step 4: Training the Agent**

- Trained the model using `model.learn()`
- Agent interacts with environment through observing state, taking action, receiving reward, Training performed for multiple timesteps (e.g., 200k–300k).

5. **Step 5: Model Saving**

- Saved trained model using `model.save()`
- Enables reuse without retraining

6. **Step 6: Evaluation and Testing**

- Tested the trained agent using `model.predict()`
- Measured performance using total reward, episode length, walking stability

7. **Step 7: Video Recording of Results**

- Used Gymnasium `RecordVideo` wrapper
- Recorded agent's walking behavior as an MP4 file
- Enabled visualization in Google Colab

8. **Step 8: Result Visualization**

- Displayed recorded video inside notebook
- Observed walking improvements after training
- Compared initial vs trained behavior

CODE

```
!pip -q install "gymnasium[box2d]" stable-baselines3[extra] pygame pyvirtualdisplay imageio imageio-ffmpeg
```

```
import os
import glob
import warnings
warnings.filterwarnings("ignore")

import gymnasium as gym
from gymnasium.wrappers import RecordVideo

from stable_baselines3 import PPO
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.evaluation import evaluate_policy

from IPython.display import HTML, display
from base64 import b64encode
```

Gym has been unmaintained since 2022 and does not support NumPy 2.0 amongst other critical functionality. Please upgrade to Gymnasium, the maintained drop-in replacement of Gym, or contact the authors of your software. See the migration guide at https://gymnasium.farama.org/introduction/migration_guide/ for additional information.

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated, use datetime.datetime.now(tz=datetime.timezone.utc) instead
    return datetime.utcnow().replace(tzinfo=utc)
```

```
env = gym.make("BipedalWalker-v3")
obs, info = env.reset()

print("Observation shape:", obs.shape)
print("Action space:", env.action_space)
print("Observation space:", env.observation_space)

env.close()
```

```
<frozen importlib._bootstrap>:488: DeprecationWarning: builtin type SwigPyObject has no __module__ attribute
<frozen importlib._bootstrap>:488: DeprecationWarning: builtin type SwigPyObject has no __module__ attribute
<frozen importlib._bootstrap>:488: DeprecationWarning: builtin type swigvarlink has no __module__ attribute
/usr/local/lib/python3.12/dist-packages/pygame/pkgdata.py:25: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg\_resources.html
  from pkg_resources import resource_stream, resource_exists
/usr/local/lib/python3.12/dist-packages/pkg_resources/_init_.py:3154: DeprecationWarning: Deprecated call to `pkg_resources.declare_namespace('god')`.
Implementing implicit namespace packages (as specified in PEP 420) is preferred to `pkg_resources.declare_namespace`. See https://setuptools.pypa.io/en/latest/namespace\_packages.html
  declare_namespace(pkg)
/usr/local/lib/python3.12/dist-packages/pkg_resources/_init_.py:3154: DeprecationWarning: Deprecated call to `pkg_resources.declare_namespace('god')`.
Implementing implicit namespace packages (as specified in PEP 420) is preferred to `pkg_resources.declare_namespace`. See https://setuptools.pypa.io/en/latest/namespace\_packages.html
  declare_namespace(pkg)
/usr/local/lib/python3.12/dist-packages/pkg_resources/_init_.py:3154: DeprecationWarning: Deprecated call to `pkg_resources.declare_namespace('sp')`.
Implementing implicit namespace packages (as specified in PEP 420) is preferred to `pkg_resources.declare_namespace`. See https://setuptools.pypa.io/en/latest/namespace\_packages.html
  declare_namespace(pkg)
Observation shape: (24,)
Action space: Box(-1.0, 1.0, (4,)), float32)
Observation space: Box([-3.1415927 -5.          -5.          -5.          -3.1415927 -5.
-3.1415927 -5.          -0.          -3.1415927 -5.          -3.1415927
-5.          -0.          -1.          -1.          -1.          -1.
-1.          -1.          -1.          -1.          -1.          -1.
], [3.1415927 5.          5.          5.          3.1415927 5.          3.1415927
5.          5.          3.1415927 5.          3.1415927 5.          5.
1.          1.          1.          1.          1.          1.          1.
1.          1.          1.          ], (24,)), float32)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use datetime.datetime.now(datetime.timezone.utc) instead.
  return datetime.datetime.utcnow().replace(tzinfo=utc)
```

```
os.makedirs("models", exist_ok=True)
os.makedirs("videos", exist_ok=True)
os.makedirs("logs", exist_ok=True)

train_env = gym.make("BipedalWalker-v3")
train_env = Monitor(train_env)

model = PPO(
    policy="MlpPolicy",
    env=train_env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    verbose=1,
    tensorboard_log="./logs/"
)

Using cpu device
Wrapping the env in a DummyVecEnv.
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version
    return datetime.utcnow().replace(tzinfo=utc)
```

```
model.learn(total_timesteps=10000)
model.save("models/ppo_bipedalwalker_test")

Logging to ./logs/PPO_1

| rollout/ | |
| ep_len_mean | 1.6e+03 |
| ep_rew_mean | -117 |
| time/ | |
| fps | 602 |
| iterations | 1 |
| time_elapsed | 3 |
| total_timesteps | 2048 |
|-----|

| rollout/ | |
| ep_len_mean | 690 |
| ep_rew_mean | -107 |
| time/ | |
| fps | 542 |
| iterations | 2 |
| time_elapsed | 7 |
| total_timesteps | 4096 |
| train/ | |
| approx_kl | 0.011423402 |
| clip_fraction | 0.116 |
| clip_range | 0.2 |
| entropy_loss | -5.66 |
| explained_variance | -0.26 |
| learning_rate | 0.0003 |
| loss | 0.335 |
| n_updates | 10 |
| policy_gradient_loss | -0.0129 |
| std | 0.992 |
| value_loss | 0.742 |
|-----|
```



```
model.learn(total_timesteps=200000)
model.save("models/ppo_bipedalwalker_final")
```

Logging to ./logs/PPO_2

rollout/	
ep_len_mean	1.6e+03
ep_rew_mean	-102
time/	
fps	641
iterations	1
time_elapsed	3
total_timesteps	2048

rollout/	
ep_len_mean	1.09e+03
ep_rew_mean	-102
time/	
fps	553
iterations	2
time_elapsed	7
total_timesteps	4096
train/	
approx_kl	0.009908794
clip_fraction	0.114
clip_range	0.2
entropy_loss	-5.58
explained_variance	0.376
learning_rate	0.0003
loss	0.0639
n_updates	60
policy_gradient_loss	-0.0107
std	0.971
value_loss	0.527

```
video_files = glob.glob("videos/*.mp4")
video_files = sorted(video_files, key=os.path.getmtime)

latest_video = video_files[-1]
print("Showing:", latest_video)

mp4 = open(latest_video, "rb").read()
data_url = "data:video/mp4;base64," + b64encode(mp4).decode()

display(HTML(f"""
<video width=700 controls>
  <source src="{data_url}" type="video/mp4">
</video>
"""))
```

... Showing: videos/bipedal_demo-episode-0.mp4



```

test_env = gym.make("BipedalWalker-v3")
obs, info = test_env.reset()

done = False
truncated = False
episode_reward = 0
steps = 0

while not (done or truncated):
    action, _ = model.predict(obs, deterministic=True)
    obs, reward, done, truncated, info = test_env.step(action)
    episode_reward += reward
    steps += 1

test_env.close()

print("Test episode reward:", episode_reward)
print("Steps survived:", steps)

```

```

*** Test episode reward: -43.483616
Steps survived: 569

```

```

from stable_baselines3 import PPO

# Continue training with better settings

model = PPO(
    "MlpPolicy",
    train_env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    ent_coef=0.001, # helps exploration
    verbose=1,
    tensorboard_log="./logs/"
)

```

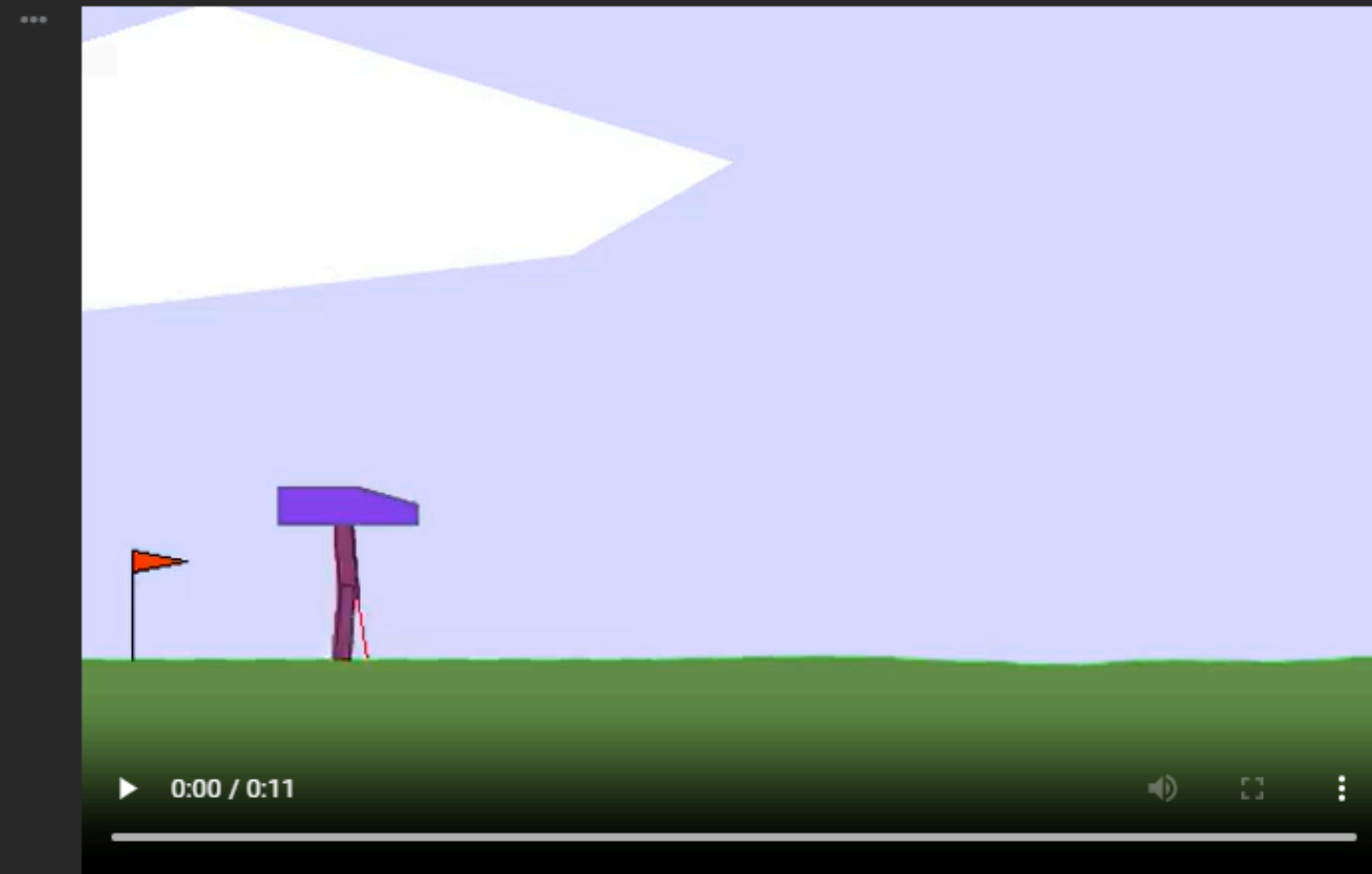
```

video_files = sorted(glob.glob("videos/*.mp4"), key=os.path.getmtime)
latest_video = video_files[-1]

mp4 = open(latest_video, "rb").read()
data_url = "data:video/mp4;base64," + b64encode(mp4).decode()

display(HTML(f"""
<video width=700 controls>
  <source src="{data_url}" type="video/mp4">
</video>
"""))

```



PPO VS TD3

	Average Reward	Best Reward	Worst Reward	Reward Std	Average Steps Survived	Max Steps Survived	Min Steps Survived	Fall Rate (%)	Success Rate (%)
0	21.782314	125.910645	-79.270195	80.619263	1204.35	1600	437	55.0	0.0

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use datetime.datetime.now(tz=datetime.timezone.utc) instead.
return datetime.datetime.utcnow().replace(tzinfo=utc)

PPO ALGORITHM

	Average Reward	Episode Length (avg steps)	Walking Speed (avg)	Energy Consumption (avg)	Training Convergence (reward std)	Success Rate (%)	Fall Rate (%)	Stability Score (avg)
0	301.075256	907.85	0.010196	2245.180176	0.99874	100.0	0.0	0.06095

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use datetime.datetime.now(tz=datetime.timezone.utc) instead.
return datetime.datetime.utcnow().replace(tzinfo=utc)

TD3 ALGORITHM

RESULTS

Metric	PPO	TD3
Algorithm Type	On-Policy (Policy Gradient)	Off-Policy (Actor-Critic)
Policy Type	Stochastic (Gaussian)	Deterministic
Learning Rate	3×10^{-4}	1×10^{-3}
Batch Size	64	256
Discount Factor (γ)	0.99	0.99
Rollout / Buffer	2,048 steps (rollout)	100,000 (replay buffer)
Update Epochs	10 per rollout	1 per step
Clip Range / Twin Critics	$\epsilon = 0.2$ (clipping)	Dual Q-networks
Exploration	Stochastic policy noise	Normal noise $\sigma = 0.1$
Total Timesteps	200,000	850,000
Average Reward (20 eps)	21.78	301.07
Best Episode Reward	125.91	305
Worst Episode Reward	-79.27	298
Reward Std Deviation	80.62	0.998
Success Rate (%)	0%	100%
Fall Rate (%)	55%	0%
Avg Episode Length (steps)	1,156.75	907.85
Walking Speed (avg)	0.031	0.01
Energy Consumption (avg)	2,148.11	2,245.18
Stability Score (torso angle)	0.683	0.061
Single Recorded Episode Reward	151.9	299.43
Task Completion (reward ≥ 300)	No	Yes



THANK YOU

